Blockchain data structures

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Arnaldo Tomasino

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Core Blockchain Data Structures and Network Architecture **5**
1.1 The Anatomy of a Blockchain: Blocks and Transactions 5
1.2 Transaction Models: UTXO vs. Account-Based 5
1.3 Ensuring Data Integrity: Merkle Trees 8
1.4 Mathematical principles and implementation of the Merkle Tree 10
1.5 Network Architecture and Node Types 11
1.6 Alternative Ledger Structures: Directed Acyclic Graphs (DAGs) 12

2 Python implementation of a blockchain **15**
2.1 The Core Components: Blocks and Chains 15
2.2 Organizing Transactions: The Merkle Tree 16
2.3 Identities and Wallets 20
2.4 Representing Ownership: The UTXO Model and Transaction Anatomy 20
2.4.1 The Anatomy of a Transaction 21

1 Core Blockchain Data Structures and Network Architecture

1.1 The Anatomy of a Blockchain: Blocks and Transactions

The integrity and functionality of a blockchain derive from its core data structures: the transactions that encapsulate state changes and the blocks that aggregate these transactions into a cryptographically secured, linear history. Each block acts as a container for a batch of transactions that have been validated by the network.

The structure of a single block consists of two primary components: a header and a list of transactions. (See figure 1.1)

- **Block Header:** A compact summary of the block's content, which includes several critical fields:
  - *Block Hash:* The unique cryptographic hash of the block header itself, serving as the block's identifier.
  - *Previous Block Hash:* A cryptographic hash of the preceding block's header. This is the crucial link that forms the "chain," ensuring the sequence and integrity of the entire ledger.
  - *Timestamp:* The time at which the block was created, providing a chronological order to the ledger.
  - *Nonce:* A value used in the proof-of-work calculation. Miners repeatedly change the nonce to find a block hash that meets the network's difficulty requirements.
  - *Merkle Root:* The root hash of the Merkle tree, which is a cryptographic summary of all the transactions included in the block.
- **List of Transactions:** The payload of the block, containing the collection of all transactions that have been validated and included.

The design of these blocks provides the foundation for the ledger's security, but the method used to structure the transactions within them is equally important, leading to two primary models for managing value and ownership.

1.2 Transaction Models: UTXO vs. Account-Based

The strategic choice of a transaction model fundamentally shapes a blockchain's capabilities, scalability, and support for complex operations like smart contracts. The method used to track ownership and transfer value dictates how balances are calculated and how transactions are processed. The Unspent Transaction Output (UTXO) model and the Account-Based model represent the two dominant paradigms in distributed ledger technology.

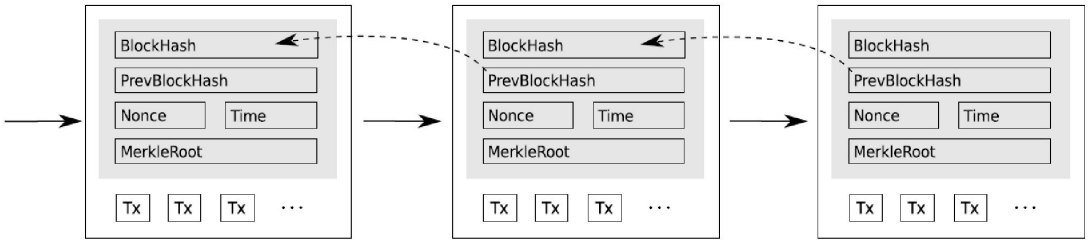

Figure 1.1: Blockchain

Feature	UTXO Model (e.g., Bitcoin)	Account-Based Model (e.g., Ethereum)
Core Concept	The ledger is a collection of unspent outputs (UTXOs) from past transactions. Ownership is tracked via these discrete pieces of value.	The ledger is a database of accounts, each with a specific state that includes a balance.
Balance Calculation	A user's balance is the sum of all UTXOs locked to their identity (address). It is not stored explicitly but calculated from available UTXOs.	A user's balance is a direct field within their account's state, which is explicitly stored and updated by transactions.
Transaction Structure	A transaction consumes one or more existing UTXOs as inputs and creates new UTXOs as outputs.	A transaction specifies a sender, a receiver, and an amount, which directly modifies the balances of the respective accounts.
Smart Contract Support	The stateless, coin-based nature of the UTXO model is ill-suited for managing the complex, multi-transactional state changes inherent to smart contracts, which are more naturally handled by the account model's explicit state transitions.	Fully supported. Transactions can be easily encoded within a smart contract to trigger complex state transitions and interactions.
Transaction Processing	Allows for parallel processing of transactions, as UTXOs are independent of one another, which can enhance scalability.	State management is generally more complex and often implemented using NoSQL databases.

In the **Unspent Transaction Output (UTXO)** model, the process of transferring value resembles handling physical cash. (See figures 1.2 and 1.3)

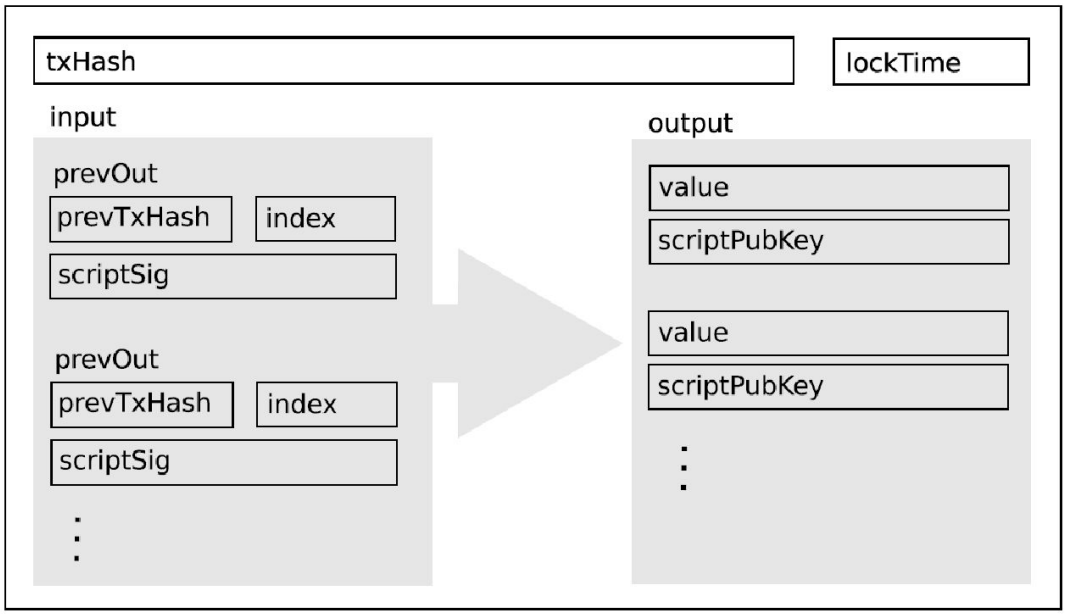

Figure 1.2: UTXO transaction structure

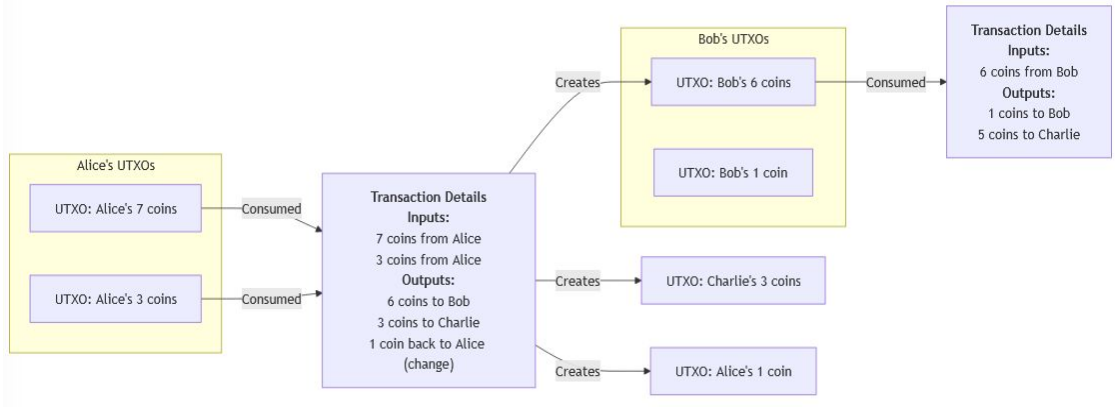

Figure 1.3: UTXO transactions example

1. A transaction consumes one or more existing UTXOs as **inputs**.
2. Each input must reference a previous, unspent output by its unique transaction hash and its index within that transaction's list of outputs.
3. The transaction then creates new **outputs**, which become new UTXOs that can be spent in future transactions.
4. Outputs cannot be split. An entire UTXO must be consumed as an input. If the value of the UTXO is greater than the amount being sent, the remainder is sent back to the sender in a new "change" output.
5. The total balance of a user is the sum of all UTXOs locked to their identity. This collection of all spendable outputs across the network is known as the **UTXO set**.

The **transaction fee** in the UTXO model is not an explicit field but is implicitly calculated as:

$$Fee = Sum(Inputs) - Sum(Outputs)$$

This remainder, which is not assigned to any specific output, is collected by the network participants (miners) as a reward for validating the transaction and including it in a block.

Security in the UTXO model is managed by "locking" and "unlocking" outputs. The most common mechanism is the **Pay-to-Public-Key-Hash (P2PKH)** script.

- An output is "locked" with a script containing the cryptographic hash of the recipient's public key.
- To "unlock" and spend this output in a new transaction, the recipient must provide two pieces of information in the transaction's input: their original public key and a valid digital signature created with their corresponding private key.
- The network verifies the transaction by first checking that the provided public key hashes to the same value contained in the locking script. It then uses the public key to verify the digital signature. This process proves ownership and authorizes the spending of the UTXO without ever exposing the user's private key. This verification is not a monolithic check but a sequence of operations executed by the network using script opcodes, such as `OP_HASH160` to hash the provided public key and `OP_EQUALVERIFY` to compare it against the lock, followed by `OP_CHECKSIG` to validate the digital signature.

While transaction models define how value is transferred, efficiently verifying the integrity of thousands of transactions within a single block requires another critical data structure: the Merkle tree.

1.3 Ensuring Data Integrity: Merkle Trees

A single block in a blockchain can contain thousands of transactions, presenting a significant challenge: how can the network efficiently verify the integrity of this large dataset without processing every single transaction? The Merkle tree, also known as a hash tree, provides an elegant solution to this problem by creating a single, compact cryptographic summary of all transactions in a block. (See figure 1.4)

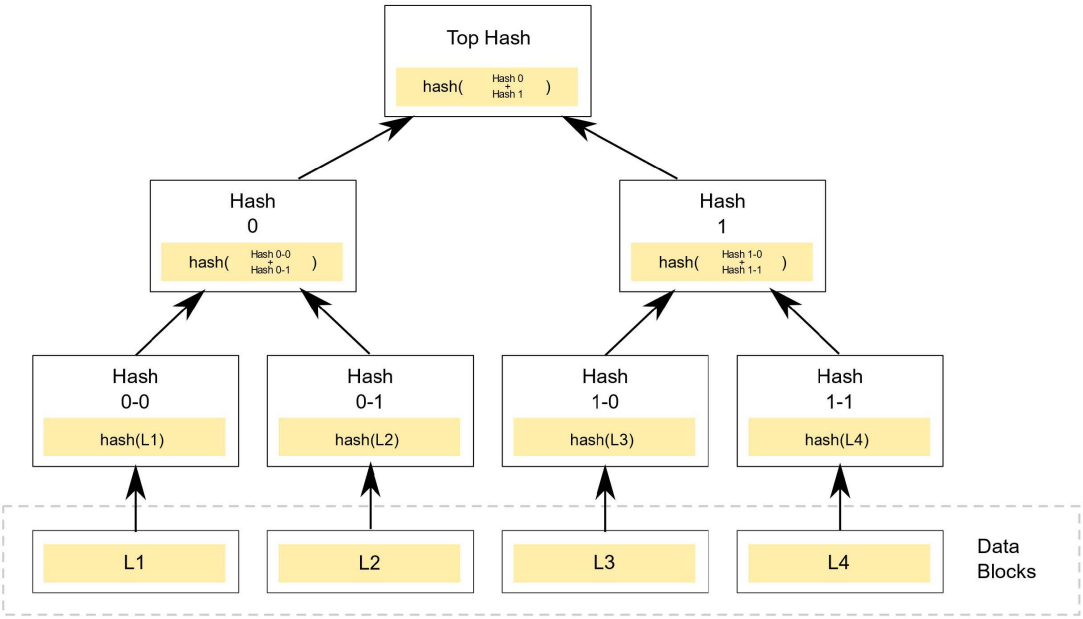

Figure 1.4: Merkle tree

The construction of a Merkle tree is a recursive hashing process:

1. Each individual transaction in the block is cryptographically hashed. These hashes form the "leaf" nodes at the bottom of the tree.
2. Adjacent pairs of leaf nodes are concatenated and hashed together, creating a parent node on the level above.
3. If there is an odd number of nodes at any level, the last node is duplicated and then hashed with itself to maintain a balanced tree structure.
4. This process of pairing, concatenating, and hashing is repeated level by level, moving up the tree until only a single hash remains. This final hash is the **Merkle Root**.

Only this single Merkle Root is stored in the block header. This design has a profound impact on data integrity: if even a single character in any transaction is altered, its corresponding leaf hash will change. This change will cascade up the tree, altering every parent hash along its path and ultimately resulting in a completely different Merkle Root. This change would invalidate the block header and, by extension, the entire block, thus securely connecting the header to every transaction within it.

Furthermore, Merkle trees enable highly efficient **Transaction Verification using Merkle Proofs** (See figure 1.5). To verify if a specific transaction is included in a block, a user does not need to download and process all transactions. Instead, they only need a **Merkle Proof**, which consists of the transaction's hash and the chain of sibling hashes along the path from the

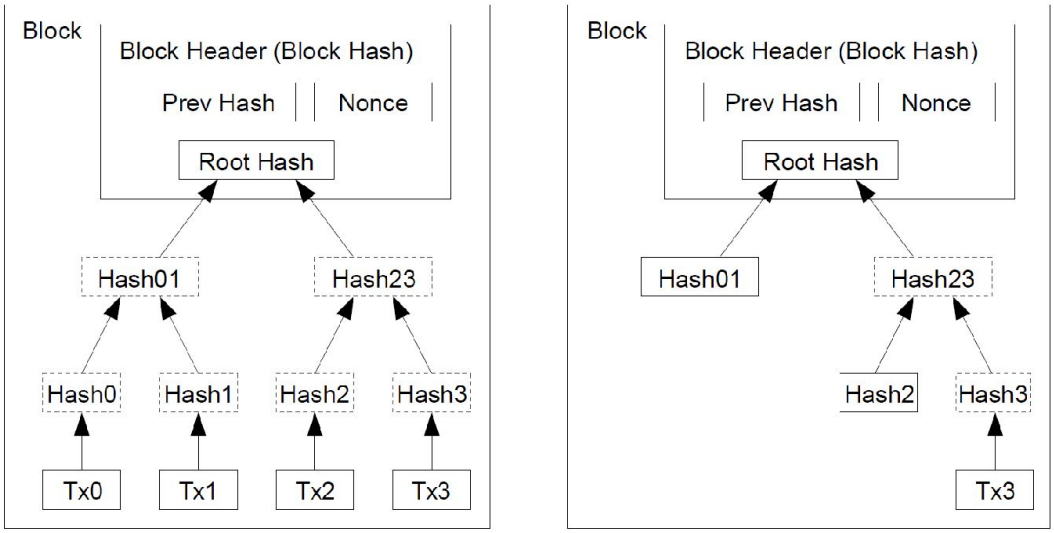

Figure 1.5: Merkle proof

transaction's leaf node to the root. To verify, a lightweight node takes the hash of the transaction in question, concatenates it with the first sibling hash provided in the proof, and hashes the result. This new hash is then concatenated with the next sibling hash at the level above, and the process is repeated up the tree. If the final computed hash matches the Merkle Root stored in the block header, the transaction's inclusion is cryptographically proven. This allows for verification with a number of computations proportional to the logarithm of the number of transactions (O(log n)), making it incredibly efficient.

This efficiency of Merkle proofs is not merely theoretical; it is the fundamental mechanism that enables the tiered network architecture, allowing Lightweight Nodes to securely verify transactions without the immense storage overhead of an Archival Full Node.

1.4 Mathematical principles and implementation of the Merkle Tree

The structure of a balanced binary Merkle Tree follows a clear mathematical relationship. The number of leaves (y), which corresponds to the number of transactions, and the tree's depth (d), which is the number of levels, are related by the following formulas:

$$y = 2^d$$
$$d = log_2(y)$$

This elegant relationship, however, introduces an implementation challenge: the number of transactions in a block is rarely a perfect power of two. To construct a balanced binary tree (See figure 1.6), the set of leaf nodes must be adjusted through a specific balancing process:

1. **If the number of leaves is odd:** Duplicate the final leaf node and append it to the list. Repeat this process until the total number of leaves is a power of two.

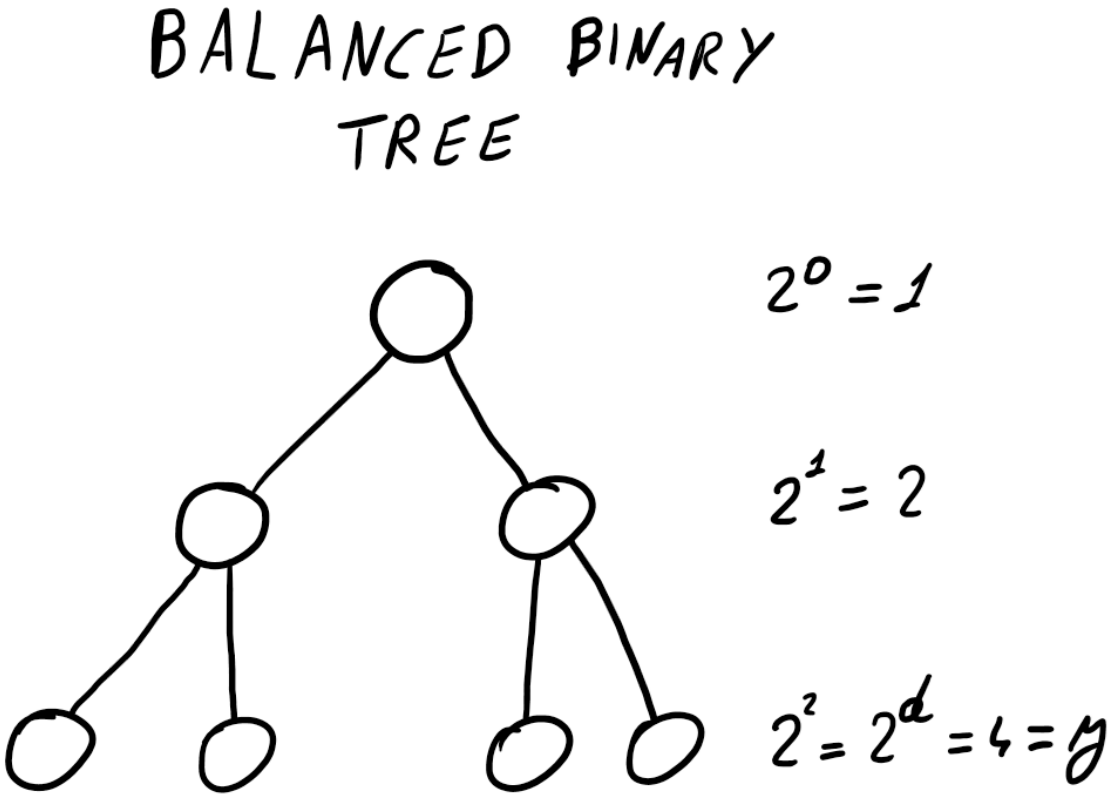

Figure 1.6: Balanced binary tree

2. **If the number of leaves is even (but not a power of two):** Take the final two leaf nodes in the list and append duplicates of them (as a pair) to the list. Repeat this until the total number of leaves is a power of two.

This two-step procedure ensures the set of leaves can always be symmetrically paired from the bottom up, allowing a complete and balanced Merkle Tree to be constructed for any number of transactions.

1.5 Network Architecture and Node Types

A blockchain is not a monolithic entity but a decentralized network of communicating computers, or nodes, each running the blockchain's software. To create an efficient and scalable network, not all nodes are required to perform the same functions. They can be categorized based on the amount of data they store and the validation tasks they perform, resulting in a tiered architecture that accommodates a wide range of participants.

- **Full Node:** A full node is an active participant in the network that enforces the system's rules. It downloads and validates every block and transaction against the blockchain's consensus protocol. By doing so, full nodes maintain the security and integrity of the network without having to trust any other participant. There are two main types of full nodes:

  - *Archival Node:* This is a full node that downloads and stores a complete, historical copy of the entire blockchain from the very first block to the present. Archival nodes serve as a backbone for the network, providing a data source for new nodes that need to sync with the blockchain's history.
  - *Pruned Node:* This is a full node that also validates all new data but discards older blocks after validation to save disk space. While it doesn't keep the full transaction history, it maintains the complete UTXO set in a separate, in-memory database (often called the `chainstate`). Because this set represents the complete collection of all currently spendable outputs, it is sufficient to validate all new transactions and blocks without requiring the full historical block data.

- **Lightweight Node (or Thin Client):** A lightweight node is designed for users who need to interact with the blockchain without the significant storage and bandwidth requirements of running a full node. It does not store the entire blockchain; instead, it downloads only the block headers. To verify if a specific transaction is included in a block, it relies on full nodes to provide a Merkle Proof. While this approach is highly efficient, it introduces an element of trust, as the lightweight node must assume that the full nodes providing the information are honest.

This network architecture, built around a linear chain of blocks, is the most established model for distributed ledgers. However, alternative architectures have emerged to address different technical challenges and use cases.

1.6 Alternative Ledger Structures: Directed Acyclic Graphs (DAGs)

As distributed ledger technologies (DLTs) have evolved, new data structures have been proposed to overcome the scalability, latency, and transaction fee limitations of traditional blockchains. These challenges are particularly acute in high-throughput applications like the Internet of Things (IoT). The **Directed Acyclic Graph (DAG)** has emerged as a prominent alternative ledger structure designed for these environments. (See figure 1.7)

Unlike a blockchain, which bundles transactions into blocks that are added sequentially, a DAG-based DLT allows transactions to be added directly to the graph structure. The validation mechanism is also different. Instead of relying on miners and a global consensus protocol, a DAG requires that for a new transaction to be accepted into the ledger, it must first approve two or more previous transactions. In this model, each transaction participates in securing the network, creating a flow of approvals rather than a chain of blocks.

The structural differences between blockchains and DAGs result in distinct performance and operational characteristics. (See table 1.1)

The architecture of a distributed ledger—from its transaction models and data integrity mechanisms like Merkle trees to its overarching structure as a linear chain or a directed graph—is a series of deliberate design choices. These choices collectively determine the ledger's performance, security, and ultimate suitability for specific applications, reflecting a dynamic and evolving technological landscape.

Metric	Traditional Blockchain	DAG-Based DLT
Throughput (transactions/second)	Lower	Higher
Power Consumption	Higher	Lower
Transaction Latency	Higher	Lower
Scalability	Lower	Higher
Transaction Fees	Higher	Lower
Consensus Robustness	Higher	Lower
Popularity	Higher	Lower

Table 1.1: Traditional blockchain and DAG-based DLT comparison

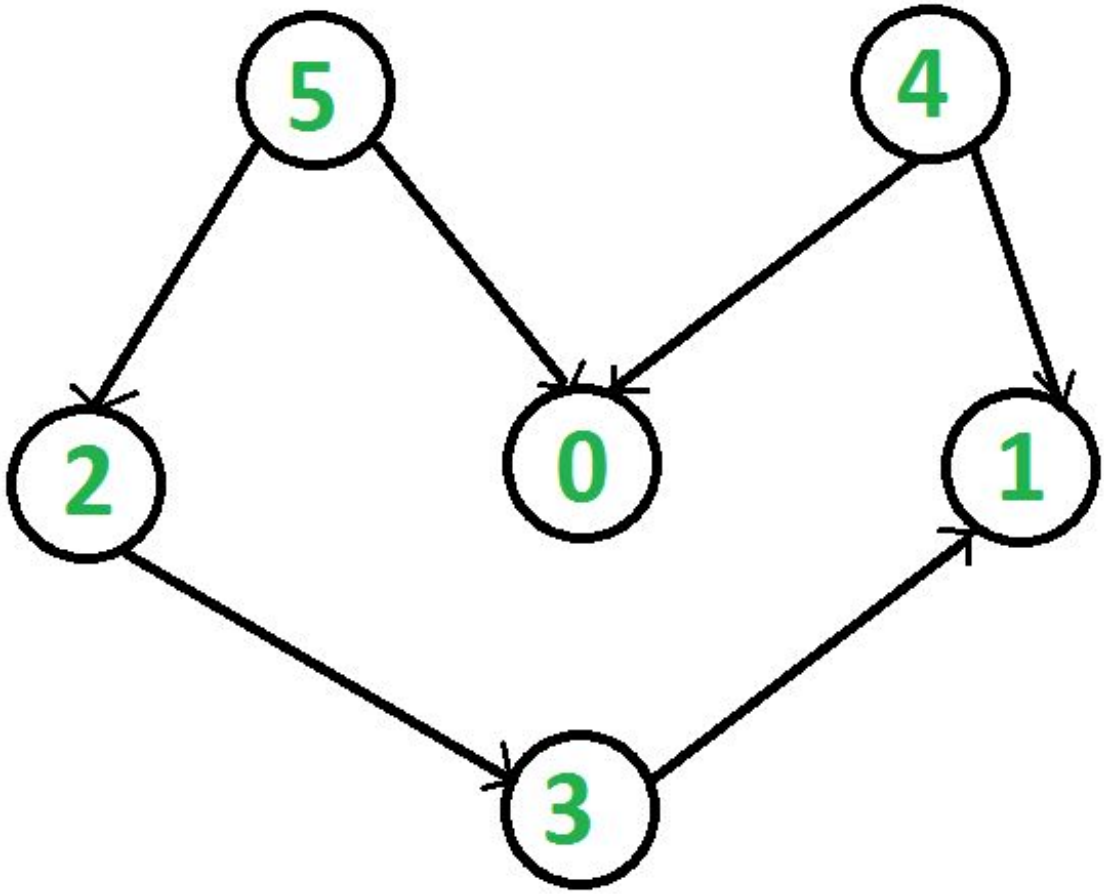

Figure 1.7: Directed Acyclic Graph

2 Python implementation of a blockchain

In this chapter we will expose a recap of the previously exposed topics and the Python codes used to develop a blockchain. These codes are just to have a better understanding of the topic and **they will not be in the exam** at least in the A.Y. 2025-2026.

2.1 The Core Components: Blocks and Chains

At its core, a blockchain is a distributed, immutable ledger. It consists of a growing list of records, called blocks, which are linked together using cryptography. Each block in the chain is a discrete data structure, and while implementations may vary, they generally contain the following core components (See figure 2.1):

- **Timestamp:** A record of when the block was created, providing a chronological anchor.
- **Transaction Data:** The payload of the block, which typically contains a set of validated transactions. In many systems, this data is organized into a sophisticated data structure called a Merkle Tree.
- **A cryptographic hash of the previous block:** This is the crucial element that connects one block to the previous.

The inclusion of the *previous block's hash* is what creates the "chain." This design, often implemented using an object-oriented approach, forges a link that is both chronological and cryptographically secure (See figure 2.2). Each new block points back to the one that came before it by containing a unique and tamper-evident fingerprint of that preceding block. Any attempt to alter the data in a past block would change its hash, which would in turn invalidate the hashes of all subsequent blocks, making such tampering immediately obvious to the entire network.

This immutable linkage is made possible by cryptographic hash functions, with `SHA-256` being a common standard. A hash function takes an input of any size and produces a fixed-size string

```
class Block:
    gbelange
    def __init__(
            self,
            timestamp: float,
            transaction_data: str,
            previous_block=None,
    ):
        self.transaction_data = transaction_data
        self.timestamp = timestamp
        self.previous_block = previous_block
```

Figure 2.1: Python block class

```python
@property
def previous_block_cryptographic_hash(self):
    previous_block_cryptographic_hash = ""
    if self.previous_block:
        previous_block_cryptographic_hash = self.previous_block.cryptographic_hash
    return previous_block_cryptographic_hash


@property
def cryptographic_hash(self) -> str:
    block_content = {
        "transaction_data": self.transaction_data,
        "timestamp": self.timestamp,
        "previous_block_cryptographic_hash": self.previous_block_cryptographic_hash
    }
    block_content_bytes = json.dumps(block_content, indent=2).encode('utf-8')
    return calculate_hash(block_content_bytes)
```

Figure 2.2: Chain creation through the inclusion of the previous hash in the block

```python
def calculate_hash(data) -> str:
    if type(data) == str:
        data = bytearray(data, "utf-8")
    h = SHA256.new()
    h.update(data)
    return h.hexdigest()
```

Figure 2.3: Hash computation function

of characters, known as a hash. The process is deterministic (the same input always produces the same output) but irreversible. `SHA-256` generates a unique, 256-bit hash for the contents of each block, ensuring the integrity and immutability of the entire chain. (See figure 2.3)

While the chain provides the crucial linkage *between* blocks, the transaction data *within* each block also requires a sophisticated structure to ensure efficiency and enable rapid verification.

2.2 Organizing Transactions: The Merkle Tree

The standard method for aggregating transactions within a block is a data structure known as the **Merkle Tree**. Its use is of strategic importance, as it provides a solution for achieving efficiency, security, and scalability in a distributed network where not all participants may have the resources to store the entire history of transactions.

A Merkle Tree is a balanced binary tree in which every leaf node contains the cryptographic hash of a single data block (in this case, a transaction), and every non-leaf node contains the cryptographic hash of the concatenated hashes of its two child nodes. This process continues up the tree until a single hash, the "Merkle root", is generated at the top. This root hash serves as a compact and secure summary of all the transactions included in the block. To make sure that the binary tree is balanced, the generation of the tree needs to address the cases in which the number of leaves is odd or even but not a power of 2 through a self balancing process.

Now, let us expose the Python implementation of the Merkle Tree referring to the file merkle_tree.py.

The file contains a class with only the constructor method: the Node class.

```
class Node:
    def __init__(self, value: str, left_child=None, right_child=None):
        self.value = value
        self.left_child = left_child
        self.right_child = right_child
```

Listing 2.1: Node class

Each node contains:

- A cryptographic hash value (called "value" in the code).
- A reference to a left child node.
- A reference to a right child node.

The node references have a default value of `None` to address the cases of the leaf nodes which do not have child elements and also to make the class easier to use later.

Before we start actually building the Merkle Tree, we need to implement a balancing process to make sure that the resulting number of leaves is a perfect power of 2.

We implement the following two utility functions to use them later in the tree balancing function.

```
def compute_tree_depth(number_of_leaves: int) -> int:
    return math.ceil(math.log2(number_of_leaves))


def is_power_of_2(number_of_leaves: int) -> bool:
    return math.log2(number_of_leaves).is_integer()
```

Listing 2.2: Utility functions

To understand the `compute_tree_depth` function we need to recall the mathematical relationship between the number of leaf nodes (y) and the depth of the tree (d).

$$y = 2^d$$
$$d = log_2(y)$$

The `compute_tree_depth` uses the second relationship and calculate its ceiling approximation in case the depth is not an integer value. For example, let's say that we have 7 transactions in the block. We have that $log_2(7) \approx 2.8$. We expect the balancing process to add a leaf node to reach 8 which is a perfect power of 2. So, after the balancing process, we will have $log_2(8) = 3$ which is exactly $\lceil log_2(7) \rceil$. In conclusion, the ceiling approximation is to calculate the depth of the resulting balanced binary tree after the balancing process.

The function `is_power_of_2` calculates the logarithm of base 2 of the number of leaves and checks if it is a perfect integer value.

Now we can move on to the actual balancing function (The original function uses identifiers referring to nodes, but they are meant to be hashes. In fact, the Merkle Tree building function will use the data as the values of the nodes which means that they are hashes, not nodes.):

```
def fill_set(list_of_hashes: list):
    current_number_of_hashes = len(list_of_hashes)

    # In this case we don't need to do anything
    if is_power_of_2(current_number_of_hashes):
        return list_of_hashes

    # We calculate the target number of leaves y as 2^d
    total_number_of_hashes =
        2 ** compute_tree_depth(current_number_of_hashes)

    # We add leaf nodes to the tree distinguishing
    # the two imbalanced tree cases.

    # The number of leaves is even but not a power of 2
    if current_number_of_hashes % 2 == 0:
        for i in range(current_number_of_hashes,
                       total_number_of_hashes, 2):
            list_of_hashes =
                list_of_hashes + [list_of_hashes[-2],
                    list_of_hashes[-1]]

    # The number of leaves is odd
    else:
        for i in range(current_number_of_hashes,
                total_number_of_hashes):
            list_of_hashes.append(list_of_hashes[-1])

    return list_of_hashes
```

Listing 2.3: Tree balancing function

The balancing function takes in input the list of hashes and calculates its length as a first operation. Before we proceed, we need to make sure that the number of hashes is not already a perfect power of 2 because, in such case, we don't have to do anything and we can return the list of hashes as is.

If the number of hashes is not a perfect power of 2, we calculate the target number of hashes using the relation $y = 2^d$ and the `compute_tree_depth` function. Now that we have the target number of hashes we need to add hashes to the list to reach the target length. We recall that the balancing process consists of the following operations:

1. **If the number of leaves is odd:** Duplicate the final leaf node and append it to the list. Repeat this process until the total number of leaves is a power of two.
2. **If the number of leaves is even (but not a power of two):** Take the final two leaf nodes in the list and append duplicates of them (as a pair) to the list. Repeat this until the total number of leaves is a power of two.

Now, let us expose the actual Merkle Tree building function.

```
def build_merkle_tree(node_data: [str]) -> Node:
    complete_set = fill_set(node_data)
    old_set_of_nodes = [Node(calculate_hash(data)) for data in complete_set]
    tree_depth = compute_tree_depth(len(old_set_of_nodes))

    for i in range(0, tree_depth):
        num_nodes = 2**(tree_depth-i)
        new_set_of_nodes = []
        for j in range(0, num_nodes, 2):
            child_node_0 = old_set_of_nodes[j]
            child_node_1 = old_set_of_nodes[j+1]
            new_node = Node(
                value=calculate_hash(f"{child_node_0.value}{child_node_1.value}"),
                left_child=child_node_0,
                right_child=child_node_1
            )
            new_set_of_nodes.append(new_node)
        old_set_of_nodes = new_set_of_nodes
    return new_set_of_nodes[0]
```

Listing 2.4: Function to build the Merkle Tree

This function takes the list of hashes in input and returns the Merkle Root as the output. As a first operation, we make sure that the length of the list of hashes is a power of two using the `fill_set` function. Once we have an appropriate list of hashes, we can use it to create a list of nodes and we can use the length of the list of nodes to calculate the depth of the Merkle Tree.

At this point we can use a for loop to generate the tree from the bottom to the top. First, we calculate the number of nodes at the current level of the tree and we initialize the list of nodes at the higher level as an empty list. Then, we use a nested for loop to retrieve the nodes at the current level two at a time and generate their parent node. We set the references to the child nodes and we use the concatenation of their hash values to calculate the hash value of the parent. Once we have the parent node, we append it to the list of nodes at the higher level.

After the inner for loop, the value of the new set of nodes becomes the value of the old set of nodes for the following iteration. And after the outer for loop we can retrieve the Merkle Root as the first (only) node contained in the list of nodes at the higher level.

To have a better understanding of the function, let us apply it to the balanced binary tree we exposed in section 1.4. (See figure 1.6)

The list of hashes contains 4 elements and 4 is a power of 2. This means that, in this case, the variable `complete_set` is equal to the variable `node_data` because the function `fill_set` does not add elements. We generate the list of nodes and we compute $d = log_2(y) = 2$.

As for the outer loop, we have that `range(0, tree_depth) = [0, 1]` so, we have 2 iterations:

1. In this iteration we have that $i = 0$ and $num_nodes = 2^{(d-0)} = 2^2 = 4$. This means that the inner for loop iterates two times to calculate the nodes at the higher level.

```python
class Owner:
    gbelange *
    def __init__(self,
                 private_key: RSA.RsaKey,
                 public_key_hash: str,
                 public_key_hex: str):
        self.private_key = private_key
        self.public_key_hash = public_key_hash
        self.public_key_hex = public_key_hex
```

Figure 2.4: Owner class

2. In this iteration we have that $i = 1$ and $num_nodes = 2^{(d-1)} = 2^1 = 2$. This means that the inner for loop iterates only once to calculate the Merkle Root which is in the position 0 of the list `new_set_of_nodes`.

2.3 Identities and Wallets

Now let us expose the Python implementation of the wallet that we can find in the file wallet.py. This file, in addition to the codes we are about to expose, also contains the `Transaction` class that we will expose in subsection 2.4.1.

The file starts with the definition of the `Owner` class which only has the constructor method (See figure 2.4). A user's identity on the network is defined by a set of cryptographic keys, which are typically managed by a software application known as a wallet. The core components are:

1. A `private key`, a secret number that grants the owner the authority to spend their funds.
2. A `public key`, which is mathematically derived from the private key.
3. An `address` (called `public_key_hash` in the code), which serves as the public identifier where funds can be sent.

After the `Owner` class we can find the `initialize_wallet` function (See figure 2.5). In this function we generate the values contained in the `Owner` class. We start by generating a private key using the RSA algorithm and deriving its relative public key. Then, we calculate an hexadecimal representation of the public key because the `Owner` class requires it and we convert it into a Python string using the `decode("utf-8")` method. The address is generated from the hexadecimal public key through a double-hashing process designed to provide both security and a shorter, more manageable identifier. First, the public key is hashed using the `SHA-256` algorithm which produces a digest of 64 hexadecimal characters. The result of that hash is then hashed again, this time using the `RIPEMD160` algorithm whose deigest is 40 hexadecimal characters long. This final `RIPEMD160` hash is the user's public address.

2.4 Representing Ownership: The UTXO Model and Transaction Anatomy

Unlike traditional banking systems which rely on an "account-based" ledger that tracks balances, foundational cryptocurrencies like Bitcoin introduced a different accounting method: the

```python
def initialize_wallet():
    private_key = RSA.generate(2048)
    public_key = private_key.publickey().export_key("DER")
    public_key_hex = binascii.hexlify(public_key).decode("utf-8")
    public_key_hash = calculate_hash(
        calculate_hash(public_key_hex, hash_function="sha256"),
        hash_function="ripemd160")
    return Owner(private_key=private_key,
                 public_key_hash=public_key_hash,
                 public_key_hex=public_key_hex)
```

Figure 2.5: Initialize wallet function

```python
class Transaction:
    ± gbelange
    def __init__(self, owner: Owner, inputs: [TransactionInput], outputs: [TransactionOutput]):
        self.owner = owner
        self.inputs = inputs
        self.outputs = outputs

    1 usage   ± gbelange
    def sign_transaction_data(self):
        transaction_dict = {
            "inputs": [tx_input.to_json(with_signature_and_public_key=False) for tx_input in self.inputs],
            "outputs": [tx_output.to_json() for tx_output in self.outputs]
        }
        transaction_bytes = json.dumps(transaction_dict, indent=2).encode('utf-8')
        hash_object = SHA256.new(transaction_bytes)
        signature = pkcs1_15.new(self.owner.private_key).sign(hash_object)
        return signature

    3 usages (3 dynamic)   ± gbelange
    def sign(self):
        signature_hex = binascii.hexlify(self.sign_transaction_data()).decode("utf-8")
        for transaction_input in self.inputs:
            transaction_input.signature = signature_hex
            transaction_input.public_key = self.owner.public_key_hex
```

Figure 2.6: Transaction class

Unspent Transaction Output (UTXO) model. In this model, there is no concept of an account balance. Instead, the blockchain tracks individual pieces of currency—the unspent outputs of past transactions—which are scattered across the ledger and controlled by their respective owners.

2.4.1 The Anatomy of a Transaction

A transaction is a data structure that facilitates the transfer of value on the blockchain (See figure 2.6). It is fundamentally composed of inputs and outputs. We define them in classes because each transaction object can have more than one input and output object. (See figures 2.7 and 2.8)

- **Transaction Owner:** The `Owner` object representing the identity of the user who issued the transaction.
- **Transaction Inputs:** These are pointers to UTXOs from previous transactions that the sender now wishes to spend. Each input must specify:
  - The transaction ID of a past transaction containing the UTXO (`transaction_hash` in the code).
  - The index of the specific output from that past transaction which is now being spent. (The union of the transaction ID and the output index uniquely identifies the output of the past transaction being used as input for the current one)
  - The public key of the owner of the output (To identify the user).
  - A digital signature generated with the owner's private key (To certify that the transaction was issued by the user).

```python
class TransactionInput:
    gbelange *
    def __init__(self,
                 transaction_hash: str,
                 output_index: int,
                 public_key: str = "",
                 signature: str = ""):
        self.transaction_hash = transaction_hash
        self.output_index = output_index
        self.public_key = public_key
        self.signature = signature
```

Figure 2.7: Transaction input class

```python
class TransactionOutput:
    gbelange
    def __init__(self, public_key_hash: str, amount: int):
        self.amount = amount
        self.public_key_hash = public_key_hash
```

Figure 2.8: Transaction output class

- **Transaction Outputs:** These are the new UTXOs being created by the transaction. Each output contains:
  - The value or amount of currency being transferred.
  - The public key hash (address) of the recipient, who will be the sole party able to spend this new UTXO in the future.
- **Digital sign method:** The Transaction class provides methods to digitally sign the transaction. This is the critical component that authorizes the spending of funds. To generate it, a string representation of the transaction's data (grouping its inputs and outputs, called `transaction_bytes` in the code) is created and then hashed with `SHA-256`. This hash is then "signed" using the owner's private key. This unique signature is included in each transaction input, providing cryptographic proof that the owner of the funds has authorized the transaction. Any node can verify the signature using the public key included in the input, ensuring authenticity and integrity of the transaction.

These fundamental data structures—blocks, chains, Merkle Trees, and transactions—represent the *what* of a blockchain: the data itself and how it is organized. In the next part, we will explore the *how*: the dynamic and complex process by which a global network of independent nodes agrees on the state of this shared data.